

Real-world (compare your loop to the library's) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangChain code), **Advanced** (the differences that only show up once you compare closely). Read Basic first even if you already know LangChain — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You just built the loop by hand. Now do the exact same task, but let LangChain's `AgentExecutor` run the loop for you instead. Run the same 3 questions through both, and compare: same final answers? Same number of steps?

Things to use: - `from langchain.agents import create_tool_calling_agent, AgentExecutor` - Wrap your existing Python function as a LangChain tool with the `@tool` decorator from Doc06. - `AgentExecutor(agent=agent, tools=[your_tool], max_iterations=5, verbose=True)`.

Intermediate Version

`AgentExecutor` is doing exactly the loop you just wrote — the ReAct think/act/observe cycle, a step limit, tool dispatch — just packaged as a reusable class instead of code you wrote by hand. The point of this exercise isn't "which one is better," it's that having built the primitive yourself means you can now read what `AgentExecutor` is actually doing underneath, instead of treating it as a black box.

- `create_tool_calling_agent(llm, tools, prompt)` builds the agent's "brain" — the piece that decides what to do next each round, using the same tool-calling mechanism from Doc06 underneath.
- `AgentExecutor(agent=agent, tools=tools, max_iterations=5, verbose=True)` is the loop itself — this class is your `run_agent()`, generalized to handle any number of tools and edge cases you didn't bother handling by hand.
- `verbose=True` prints each Thought/Action/Observation as it happens — put this side by side with your own `steps` list from the previous exercise's Advanced Approach, and you should see the same shape of information, just formatted differently.
- note the `max_iterations` argument — the library's version of your own step-counter loop, from the exact same "the limit isn't optional" idea in Core Concepts.

Advanced Version

Think past "does it get the same answer" — ask **does `AgentExecutor` count a "step" the same way your loop does, and does it handle a tool's exception the same way `run_tool()` does?** Some versions of `AgentExecutor` count a full think → act → observe round as 1 iteration; others

count differently. And by default, `AgentExecutor` catches an exception raised inside a tool and turns it into an "Invalid or incomplete response" message fed back to the model — similar in spirit to your own Hint 1 Advanced `run_tool()` from the Intermediate exercise, but with a *different, less specific* error string, unless you pass `handle_parsing_errors=True` and customize it. Write down, in your own words, what you'd expect to be different before reading Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the pieces and the comparison to run. Intermediate explains what each piece is actually doing, mapped directly back onto the loop you already built. Advanced asks the harder question underneath the comparison — not "does it work," but "does it count and fail the same way yours does" — which is exactly what makes reading someone else's `AgentExecutor` code later feel familiar instead of confusing.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```

same tool, wrapped for LangChain with @tool

build the AgentExecutor with that tool and a step limit

for each of your 3 test questions:
    run it through your own hand-built loop, note steps + answer
    run it through AgentExecutor, note steps + answer
    compare

```

Here's almost the whole thing — fill in your 3 questions yourself:

```

from langchain_core.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    ...

llm = ChatOpenAI(model="gpt-4o-mini")
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}"),
])
agent = create_tool_calling_agent(llm, [get_weather], prompt)
executor = AgentExecutor(agent=agent, tools=[get_weather], max_iterations=5, verbose=True)

test_questions = [...] # your 3 questions
for q in test_questions:
    my_answer = run_agent(q)
    lib_answer = executor.invoke({"input": q})
    print(my_answer, "vs", lib_answer["output"])

```

Intermediate Version

```
wrap each Doc06 tool with @tool (a thin wrapper, not a rewrite)

prompt = a ChatPromptTemplate with a system message, {input}, and {agent_scratchp
agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, max_iterations=5, verbose=True

for each test question:
    result_mine = run_agent(question)          # your own loop, from build_rea
    result_lib = executor.invoke({"input": question})
    compare result_mine.final_answer vs result_lib["output"]
    compare len(result_mine.steps) vs however many Actions verbose=True printed
```

Run this for all 3 questions, and write down anywhere the two disagreed, before checking the [Solution](#).

Advanced Version

Add one more comparison: make one of your tools raise an exception on purpose (same idea as the previous exercise's Advanced `run_tool()`), and run the *same* question through both. Watch what `AgentExecutor`'s `verbose=True` output shows for that failed step, and compare it against what your own `run_tool()` would have fed back as the Observation. Also try setting `AgentExecutor(..., max_iterations=2)` on a question you know needs 3+ steps, and compare what each one does when the limit is hit — does `AgentExecutor` raise an exception the way your `MaxIterationsExceeded` does, or does it just return whatever partial answer it has?

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get you a working side-by-side comparison on the happy path. Advanced pushes on the 2 places most real differences hide — a tool failure, and hitting the step limit — which is exactly the kind of thing that surprises people the first time they read `AgentExecutor`-based code in production and it doesn't behave quite like their own hand-built loop did.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)